Lab 4

Due

Thursday June 11th, 2026 @ midnight

Prerequisites

- A GitLab repository set up for your project. Make sure you add your team members to your project. Also, add the instructor to the project too to check your progress. It is important that the roles are all developers.
- Basic understanding of Agile methodology and sprint planning.
- Implement the **MVC pattern structure** in your Java project.

Objective

This lab will have two parts: Part 1 and Part 2.

Part 1 will guide students through a well-structured workflow for tracking progress in an Agile development environment by defining sprint backlog, milestones and issue boards.

Part 2 will ask students to create the MVC structure in your project. In the next lab assignment, we will continue to work with this pattern.

Part 1: Planning in Gitlab (30 Points)

Please follow the steps below to complete this part.

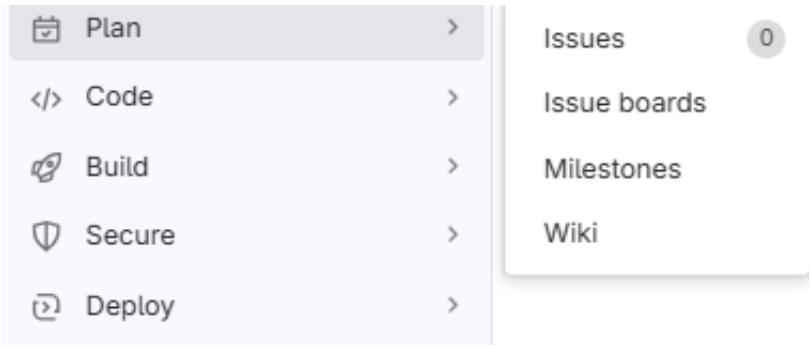
Do:

Step 1: Defining the Sprint Backlog with a Milestone

In Agile development, a sprint backlog represents the set of tasks the team commits to completing within a sprint. To effectively track and manage these tasks in GitLab, we use **milestones**. A milestone helps group related issues, providing a clear scope and deadline for the sprint. By associating issues (tasks) with a milestone, teams can ensure better visibility, track progress efficiently, and stay aligned with sprint goals.

1.1 Creating a Milestone

1. Go to Plan **>** Milestones.
2. Click New Milestone.
3. Enter a Title (e.g., 'Sprint 1'). You will be having 2 sprints. So, you need to repeat this 2 times.
4. Each sprint will be for two weeks before your lab session (the two weeks starts from June 5th, 2026). Therefore, for the “Set a Due Date” make sure it aligns with the sprint duration (two weeks each, for example June 5th - June 19th). Keep in mind that during each sprint you will showcase what you have built so far and during the last sprint review, you will be doing a demo of the whole final project (week 13).
5. Provide a Description summarizing the goal of the first sprint.
6. Click Create Milestone.



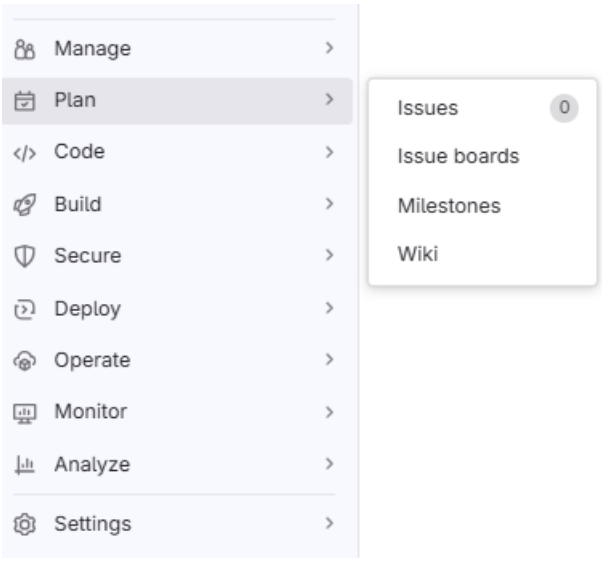
There should be at least 2 sprints created with their start dates and end dates.

Step 2: Listing All Requirements in GitLab Issues

In this step, we will add all our requirements we extracted and documented in your PRD to your Gitlab project as issues. Each issue will relate to one single requirement. Repeat the following steps for every single requirement you have defined.

2.1 Create a New Issue for Each Requirement

1. Navigate to your GitLab repository.
2. Click on Issues in the left sidebar.
3. Click New Issue.
4. Enter the Title (e.g., 'User authentication feature').
5. Provide a Description that clearly states the requirement.
6. Assign Labels such as `feature` or `bug` as needed.
7. Set an Assignee. The issues should be evenly distributed among team members, ensuring that each member has a roughly equal number of assigned issues.
8. Click Create Issue.



2.2 Organizing Issues into a Product Backlog

- Ensure all feature-related issues have a 'task', 'feature', 'non-functional', 'enhancement' or `bug` label.

For example:

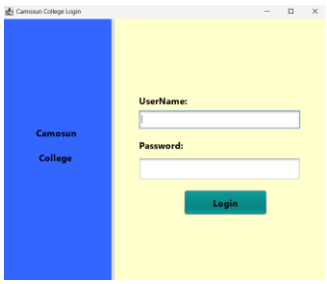
Issue Title	Label
Create Student Model	Task
Student can login (FR)	Feature
Passwords must be encrypted	Non-functional
Fix crash on registration form	Bug
Add validation messages	enhancement

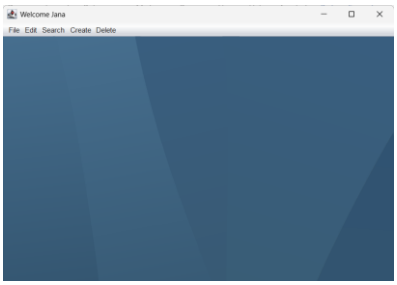
Repeat the following steps until all the requirements are added. Your requirements can have sub-tasks.

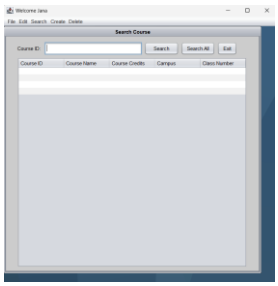
Your program will require many GUIs such as the login interface, the add/update/delete and search interfaces for each functionality per project. Start by thinking about your GUIs and adding the various GUIs as issues

to the sprint backlog. You will start with the GUIs in the next sprint but it is important to add them to the product backlog.

I have shared just a few examples of GUIs that you will work with you to give you an idea.





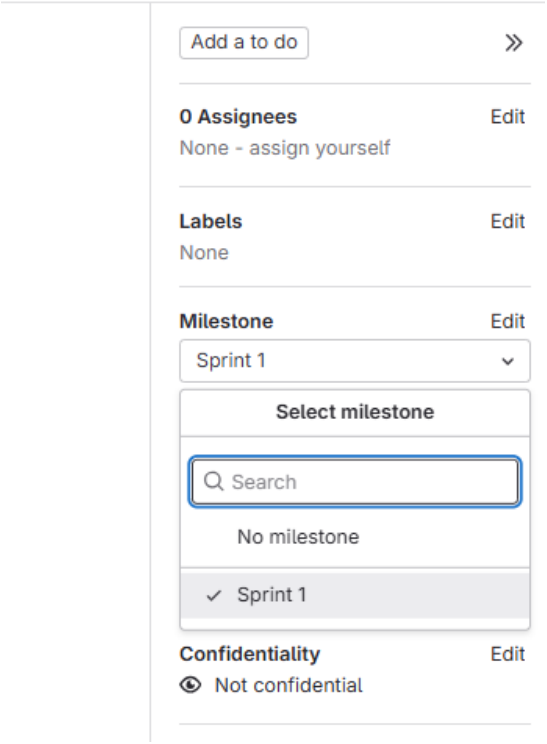


Step 3: Adding Issues to the first Sprint Backlog

Let's add issues to each sprint. For now, let's go ahead and add the high priority issues to the first sprint.

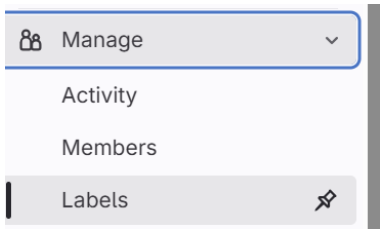
3.1 Assigning Issues to a Milestone

1. Go to Issues and open an issue that should be part of the sprint.
2. Click Edit next to Milestone field. You can find it on the right-hand side of the issue description.
3. Select the milestone (e.g., 'Sprint 1').
4. Click Save Changes.
5. Repeat for all issues that should be included in the sprint backlog.



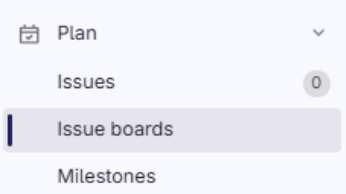
3.2 Creating Labels for Issue Board

1. Navigate to Manage > Labels. It can be found on the left column under the project.
2. Create the following labels with different colors: Sprint Backlog, To Do, In Progress, Review and Done. (Below you can find the expected output)

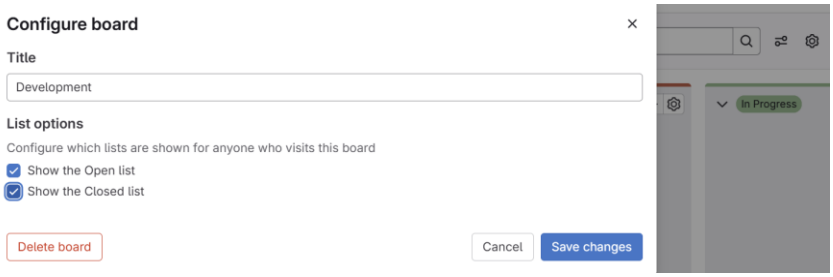


3.3 Using Boards for Sprint Backlog Management

1. Navigate to Plan > Issues Boards.



2. Click on the gear at the top-left and uncheck the “Show the Open list” and “Show the closed list”. (Screenshot is provided below).



3. Customize columns by clicking on the “New List” represent workflow stages showing: Sprint Backlog, To Do, In Progress, Review, Done.
4. Add the following tasks to this sprint backlog:
    a. Create the MVC pattern (details of this can be found in the next part).
    b. Add the JAR file
5. Whenever you are working on the issue/task assigned to you, drag the issue from the Sprint Backlog to ‘To Do’.

    Make sure the issues are assigned to one team member so they are fully aware of their tasks and will be able to start working on them as soon as possible.

    As you proceed with each issue, drag it to the next stage ‘In Progress’. Once you have completed the issue, place it in the ‘Review’ list and make sure a reviewer is assigned to your issue. The reviewer needs to check your code by bringing the changes to his local system, test it out, and if all is good, approve and place

the issue on the "Done" list. IF the changes have conflicts or bugs/errors, the reviewer is responsible for adding comments about it so that the assignee can investigate the matter and fix it. This step should be repeated for all tasks, and each person needs to follow this workflow.

6. Move issues to Done upon completion.

Part 2: Implementing MVC Structure in the Project (20 Points)

What is MVC?

Model-View-Controller (MVC) is a design pattern that separates concerns:

- **Model** → Represents the database entities and business logic.
- **View** → Handles user interaction (GUI or console output).
- **Controller** → Manages the logic and user actions.

In this lab, we will create the MVC structure within your project. Create a **utility package** that contains a DbConnection class to manage the database connection.

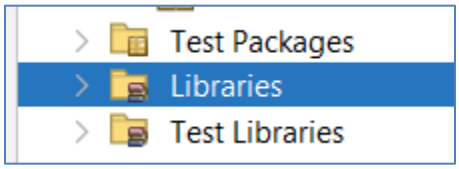
Understanding JAR Files

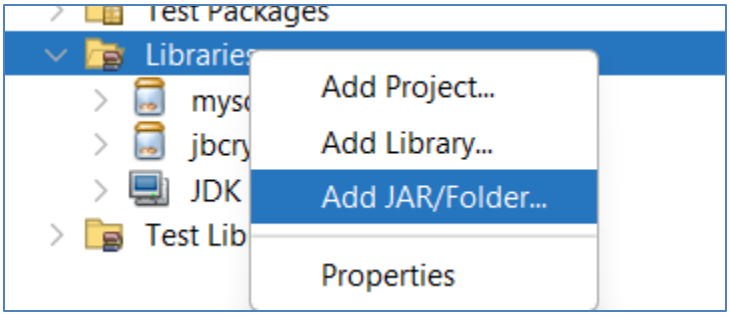
A **JAR (Java ARchive) file** is a packaged collection of **Java class files and libraries**. It allows Java applications to include dependencies like **MySQL Connector JAR** for database interaction.

Step 2.1. How to Add MySQL Connector JAR in NetBeans (5 Points)

1. Download **MySQL Connector JAR** that is shared with you within the lab assignment. Extract the zip folder.
2. In NetBeans:

    - Right-click **Libraries > Add JAR/Folder**





    - Find the mysql-connector-java-X.X.X.jar file and add it.
    - Click **OK**

Step 2.2. Creating the MVC & Utility Structure (5 Points)

In **NetBeans**, after you have cloned your project to your local system, create the following **packages** in your project:

- model → Holds the actual classes
- view → Contains GUI
- controller → Manages application logic
- dao → Data Access Objects (for database operations)
- utility → Contains DbConnection class

Make sure in the end, you do a “git add”, “git commit” and “git push”.

HAND IN

- In a word document add the link to your Gitlab project. Make sure you add me as a developer to your Gitlab project, so I can see your progress.

Evaluation Criteria:

Criteria	Marks
Part 1 - Creating the sprint	30
Part 2	20
• **Adding JAR**	5
• **Creating the MVC structure**	15
Total	**50**